Semana 2 – Normalización de salarios y valores faltantes

==========
MINERÍA DE DATOS - CLASE 2
Limpieza, Transformación y EDA
Autor: Lionel Martinez, con asistencia de IA
==========

==========
Ejercicio Sección 1 (EDA básico con clientes bancarios)
==========

```
import pandas as pd

# Crear DataFrame con los datos de clientes
df = pd.DataFrame({
    "Cliente":       [1, 2, 3, 4, 5],
    "Edad":          [25, 40, 35, 50, 30],
    "Salario":       [50000, 60000, 70000, 80000, 45000],
    "Estado_Cuenta": ["Pagado", "Moroso", "Pagado", "Moroso", "Pagado"]
})

df
```

	Cliente	Edad	Salario	Estado_Cuenta
0	1	25	50000	Pagado
1	2	40	60000	Moroso
2	3	35	70000	Pagado
3	4	50	80000	Moroso
4	5	30	45000	Pagado

==========
Markdown
Sección 2 – Limpieza, Transformación y EDA

==========

```python
import pandas as pd

df_limpieza = pd.DataFrame({
    "ID": [1, 2, 3, 4, 5],
    "Nombre": ["Juan", "María", "Pedro", "Ana", "Luis"],
    "Edad": [32, -28, 40, 35, 50],
    "Nivel_Educativo": ["Licenciado", "Ingeniero", "Doctorado",
"Licenciado", "Doctorado"],
    "Salario": [50000, 60000, None, 55000, 200000],
    "Categoria": ["Junior", "Senior", "Senior", "Junior", "Manager"],
    "Estado": ["Activo", "ACTIVO", "Inactivo", "Activo", "Inactivo"]
})

df
```

Cliente	Edad	Ingresos	Experiencia	Estado	
0	1	25	50000	2	Pagado
1	2	40	60000	10	Moroso
2	3	35	70000	7	Pagado
3	4	50	80000	15	Moroso
4	5	30	45000	3	Pagado
5	6	45	85000	20	Moroso
6	7	38	72000	12	Pagado
7	8	29	48000	4	Pagado

==========

Limpieza de datos:

En la siguiente sección se trabaja con un dataset que presenta inconsistencias, valores faltantes y outliers, con el objetivo de aplicar técnicas de limpieza y transformación de datos.

==========

```python
# 1. Corrección de edad negativa
df_limpieza["Edad"] = df_limpieza["Edad"].abs()
```

```
# 2. Imputación de valores faltantes en Salario
df_limpieza["Salario"] =
df_limpieza["Salario"].fillna(df_limpieza["Salario"].median())

# 3. Normalización de variable categórica Estado
df_limpieza["Estado"] = df_limpieza["Estado"].str.capitalize()

df_limpieza
```

	ID	Nombre	Edad	Nivel_Educativo	Salario	Categoría	Estado
0	1	Juan	32	Licenciado	50000.0	Junior	Activo
1	2	María	28	Ingeniero	60000.0	Senior	Activo
2	3	Pedro	40	Doctorado	57500.0	Senior	Inactivo
3	4	Ana	35	Licenciado	55000.0	Junior	Activo
4	5	Luis	50	Doctorado	200000.0	Manager	Inactivo

```
==============================
print(df.columns)
Index(['Cliente', 'Edad', 'Ingresos', 'Experiencia', 'Estado'],
dtype='object')
==============================
```

Se identificó un valor atípico en la variable Salario (200000), el cual fue eliminado utilizando el método IQR.

Luego de la eliminación, los datos presentan una distribución más homogénea y sin valores extremos que puedan afectar el análisis.

El valor 200000 fue identificado como un outlier extremo al superar ampliamente el límite superior definido por el método IQR, por lo que se decidió eliminarlo para evitar sesgos en el análisis.

```
==============================
df.head()
```

ID	Nombre	Edad	Nivel_Educativo	Salario	Categoria	Estado

0	1	Juan	32	Licenciado	50000.0	Junior	Activo
1	2	María	28	Ingeniero	60000.0	Senior	Activo
2	3	Pedro	40	Doctorado	57500.0	Senior	Inactivo
3	4	Ana	35	Licenciado	55000.0	Junior	Activo
4	5	Luis	50	Doctorado	200000.0	Manager	Inactivo

==========

Detección y eliminación de outliers (IQR)

==========

```
# Cálculo de cuartiles
Q1 = df["Salario"].quantile(0.25)
Q3 = df["Salario"].quantile(0.75)
IQR = Q3 - Q1

# Límites
lim_inf = Q1 - 1.5 * IQR
lim_sup = Q3 + 1.5 * IQR

# Mostrar valores (opcional pero recomendado)
print("Q1:", Q1)
print("Q3:", Q3)
print("IQR:", IQR)
print("Límite inferior:", lim_inf)
print("Límite superior:", lim_sup)

# Filtrar outliers
df = df[(df["Salario"] >= lim_inf) & (df["Salario"] <= lim_sup)].copy()

df
```

```
Q1: 55000.0
Q3: 60000.0
IQR: 5000.0
Límite inferior: 47500.0
Límite superior: 67500.0
```

ID	Nombre	Edad	Nivel Educativo	Salario	Categoria	Estado

0	1	Juan	32	Licenciado	50000.0	Junior	Activo
1	2	María	28	Ingeniero	60000.0	Senior	Activo
2	3	Pedro	40	Doctorado	57500.0	Senior	Inactivo
3	4	Ana	35	Licenciado	55000.0	Junior	Activo

==========

Prompt utilizado

ChatGPT:

"¿Cómo detectar y eliminar outliers en Python?"

Síntesis de la respuesta del LLM

El modelo sugirió utilizar métodos como el rango intercuartílico (IQR) y el Z-score para detectar valores atípicos.

En particular, el método IQR permite identificar valores fuera de un rango definido por los cuartiles (Q1 y Q3), sin asumir una distribución normal de los datos.

Validación propia

Se decidió aplicar el método IQR en este trabajo por su simplicidad y robustez frente a valores extremos. El resultado permitió identificar correctamente el valor atípico en la variable salario (200000), el cual fue eliminado para mejorar la calidad del dataset.

==========

TRANSFORMACIÓN DE DATOS

==========

```
from sklearn.preprocessing import MinMaxScaler, OrdinalEncoder

#  (Opcional pero recomendado) asegurar que no haya outliers
df_limpieza = df_limpieza[
    (df_limpieza["Salario"] >= lim_inf) &
    (df_limpieza["Salario"] <= lim_sup)
].copy()

# 1. Codificación ordinal (Nivel Educativo)
orden = [["Licenciado", "Ingeniero", "Doctorado"]]
encoder = OrdinalEncoder(categories=orden)
df_limpieza.loc[:, "Nivel_Cod"] =
encoder.fit_transform(df_limpieza[["Nivel_Educativo"]])
```

```
# 2. Normalización del salario (ya sin outliers)
scaler = MinMaxScaler()
df_limpieza.loc[:, "Salario_Normalizado"] =
scaler.fit_transform(df_limpieza[["Salario"]])

# 3. Nueva variable: años hasta jubilación
df_limpieza.loc[:, "Anios_Hasta_Jubilacion"] = 65 - df_limpieza["Edad"]

# Mostrar resultado
df_limpieza
```

ID	Nombre	Edad	Nivel_Educativo	Salario	Categoria	Estado	Nivel_Cod	Salario_Normalizado	Anios_Hasta_Jubilacion	
0	1	Juan	32	Licenciado	50000.0	Junior	Activo	0.0	0.00	33
1	2	María	28	Ingeniero	60000.0	Senior	Activo	1.0	1.00	37
2	3	Pedro	40	Doctorado	57500.0	Senior	Inactivo	2.0	0.75	25
3	4	Ana	35	Licenciado	55000.0	Junior	Activo	0.0	0.50	30

==========

Verificación de la normalización del salario

==========

```
print(df[["Salario", "Salario_Normalizado"]])
Salario  Salario_Normalizado
0  50000.0                 0.00
1  60000.0                 1.00
2  57500.0                 0.75
3  55000.0                 0.50
```

==========

El valor mínimo (50000) → 0

El valor máximo (60000) → 1

Los valores intermedios están correctamente escalados

==========

Transformaciones aplicadas

- Se realizó una **codificación ordinal** de la variable *Nivel Educativo*, asignando valores numéricos según su jerarquía (Licenciado < Ingeniero < Doctorado).

- Se aplicó **normalización Min-Max** a la variable *Salario*, escalando los valores en un rango entre 0 y 1 para facilitar comparaciones y futuros modelos.

- Se creó una nueva variable llamada **Años hasta jubilación**, calculada como la diferencia entre la edad de retiro (65 años) y la edad actual del empleado.

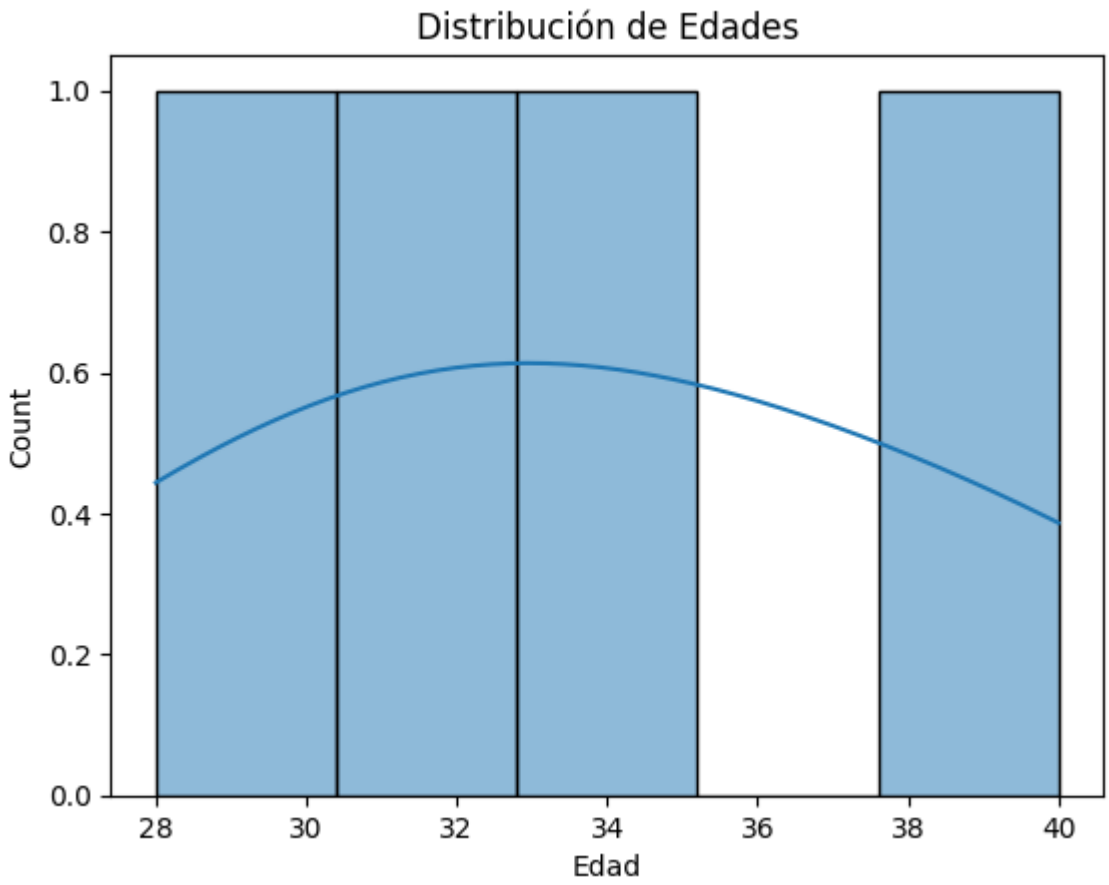
Los datos fueron correctamente transformados luego del proceso de limpieza.
Se eliminaron valores atípicos, se aplicó codificación ordinal respetando la jerarquía de la variable educativa,
y se normalizó el salario en un rango de 0 a 1.
Además, se creó la variable "Años hasta jubilación", útil para futuros análisis.

==========

Histograma de la variable Edad

==========

```
import seaborn as sns
import matplotlib.pyplot as plt

# Histograma de edades
sns.histplot(df["Edad"], bins=5, kde=True)
plt.title("Distribución de Edades")
plt.show()
```

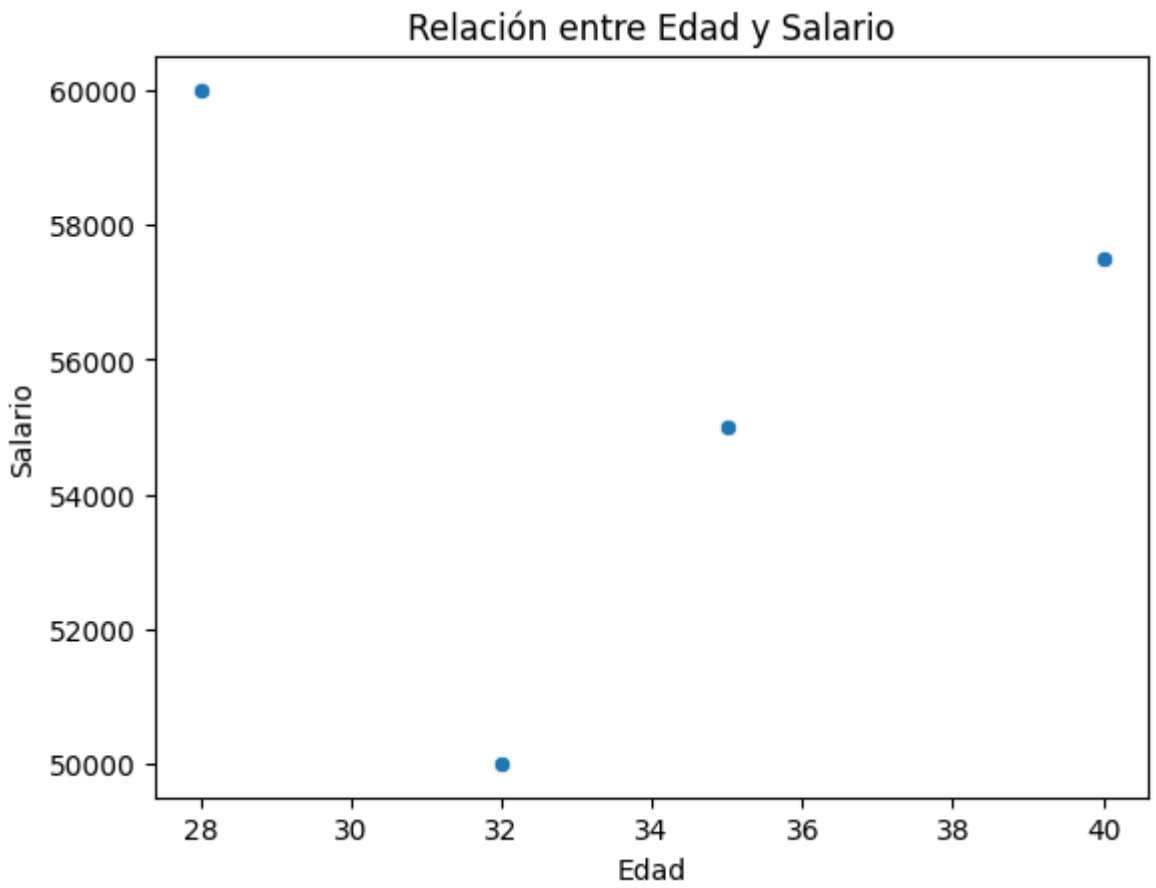


==========

Gráfico de dispersión Edad vs. Salario

==========

```
# Gráfico de relación entre Edad y Salario
sns.scatterplot(x=df["Edad"], y=df["Salario"])
plt.title("Relación entre Edad y Salario")
plt.show()
```



==========

Reflexión

Preguntas a responder:

1. ¿La variable Edad tiene una distribución normal o sesgada?
2. ¿Se observa alguna correlación entre Edad y Salario?
3. ¿Qué conclusiones preliminares sacarías antes de aplicar modelos más complejos?

==========

Interpretación del análisis exploratorio (Sección 1)

Distribución de la variable Edad:
Dado el tamaño reducido de la muestra, no es posible afirmar que la distribución siga una forma normal. Sin embargo, se observa una concentración de edades en valores

intermedios (30–40 años), con algunos valores más alejados (25 y 50), lo que indica una leve dispersión sin presencia de extremos significativos.

- **Correlación entre Edad y Salario:**
El gráfico de dispersión evidencia una tendencia positiva: a medida que aumenta la edad, también lo hace el salario. Aunque la muestra es pequeña, esta relación sugiere una posible asociación entre experiencia laboral (aproximada por la edad) y nivel salarial.

Conclusiones preliminares antes de modelar:
La variable Edad presenta cierta variabilidad, por lo que podría considerarse su normalización o estandarización en etapas posteriores, dependiendo del modelo a utilizar.

Se observa una relación positiva entre Edad y Salario, lo que indica que ambas variables podrían ser relevantes para futuros modelos predictivos o análisis de segmentación.

Este análisis exploratorio permite identificar patrones iniciales y comprender la estructura de los datos antes de aplicar técnicas más complejas.

==========

Sección 2 (Exploración de Datos Financieros con empleados)

==========

```
import pandas as pd

# Crear DataFrame con datos de empleados
df = pd.DataFrame({
    "Empleado":    [1, 2, 3, 4, 5],
    "Edad":        [25, 40, 35, 50, 30],
    "Salario":     [50000, 60000, 70000, 80000, 45000],
    "Experiencia": [2, 10, 7, 15, 3]
})

df
```

	Empleado	Edad	Salario	Experiencia
0	1	25	50000	2
1	2	40	60000	10
2	3	35	70000	7
3	4	50	80000	15

4	5	30	45000	3

==========

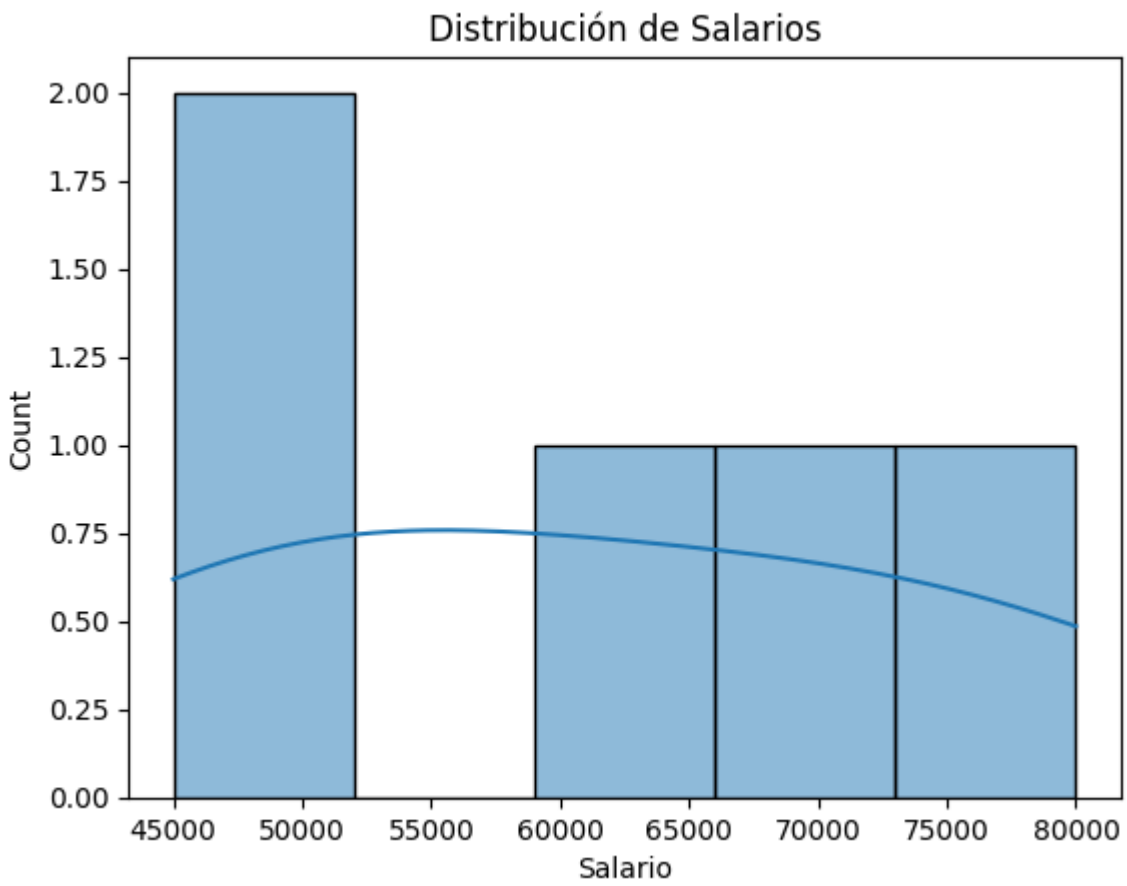
Análisis univariado del Salario

==========

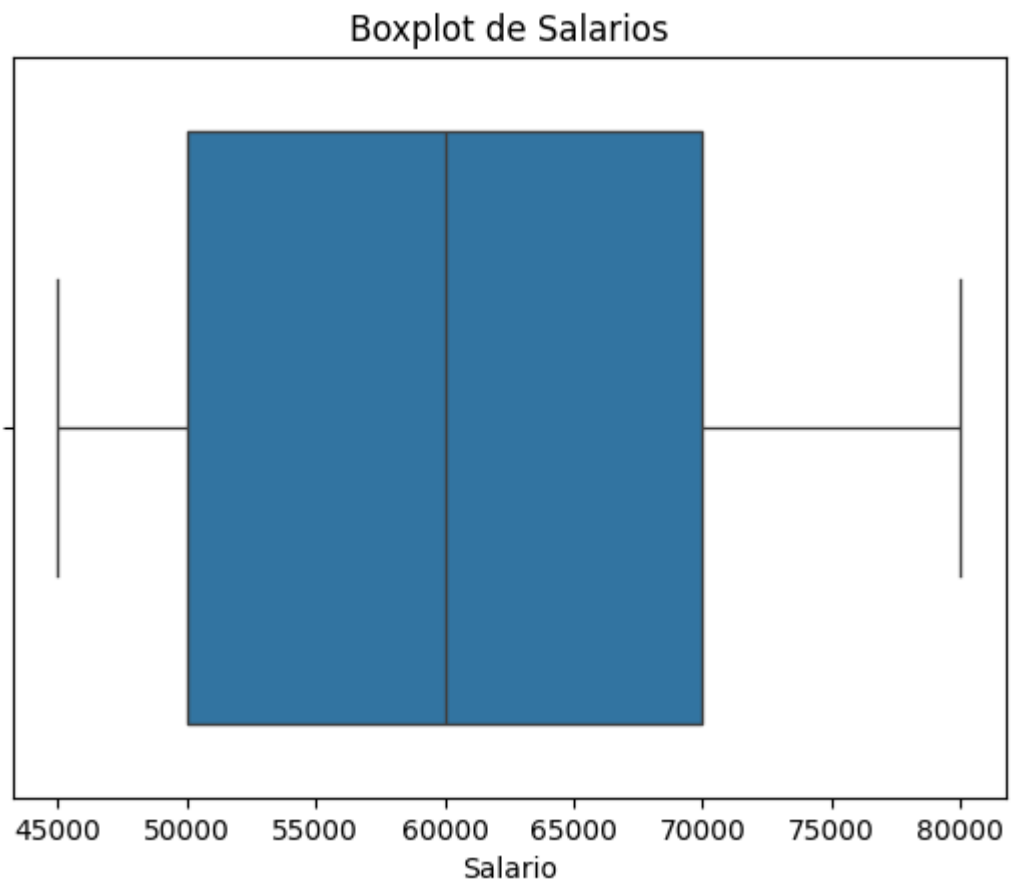
```
import seaborn as sns
import matplotlib.pyplot as plt

# Estadísticas básicas de Salario
print(df["Salario"].describe())

# Histograma de salarios
sns.histplot(df["Salario"], bins=5, kde=True)
plt.title("Distribución de Salarios")
plt.show()

# Boxplot para detectar outliers
sns.boxplot(x=df["Salario"])
plt.title("Boxplot de Salarios")
plt.show()
```

```
count        5.000000
mean     61000.000000
std      14317.821063
min      45000.000000
25%      50000.000000
50%      60000.000000
75%      70000.000000
max      80000.000000
Name: Salario, dtype: float64
```

==========


Distribución de Salarios
Count
2.00
1.75
1.50
1.25
1.00
0.75
0.50
0.25
0.00
45000
50000
55000
60000
65000
70000
75000
80000
Salario

==========

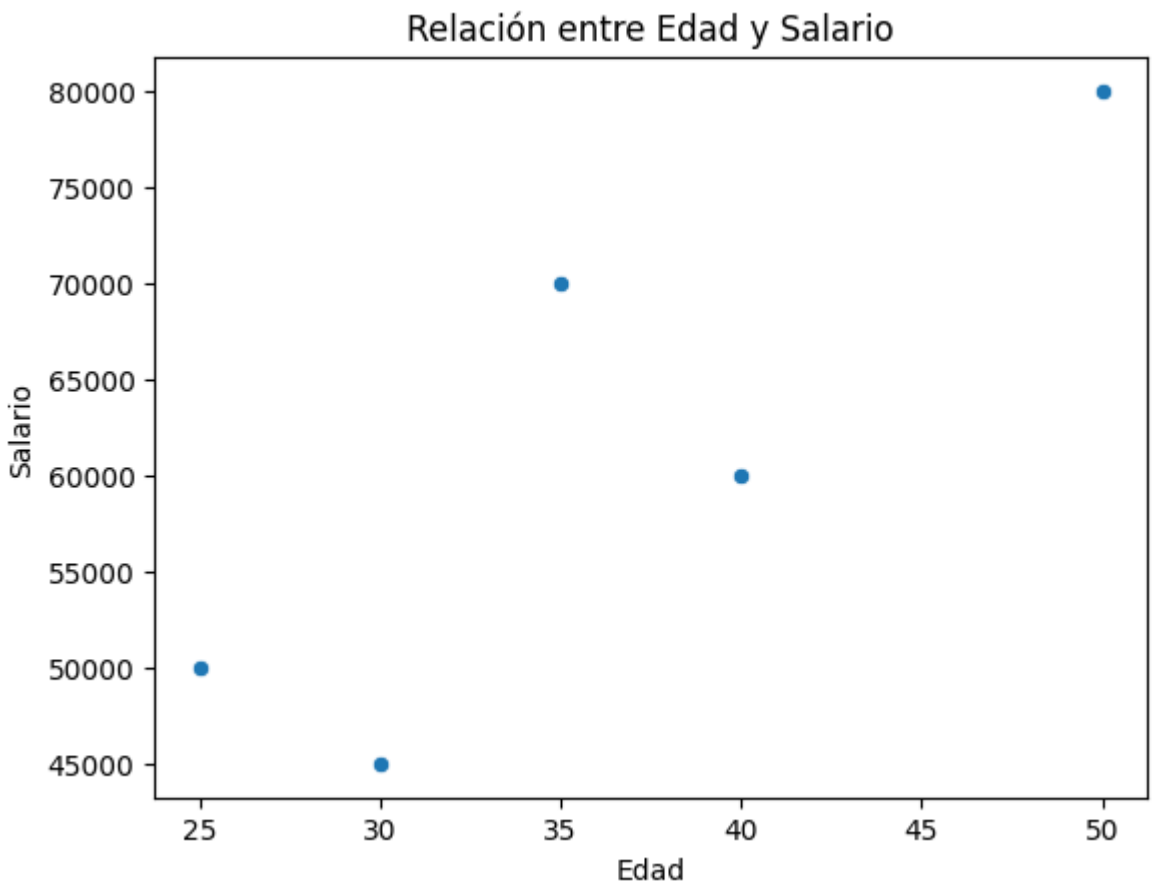


==========

Análisis multivariado (Edad vs. Salario)

==========

```
# Correlación entre Edad y Salario
print(df[["Edad", "Salario"]].corr())

# Gráfico de dispersión
sns.scatterplot(x=df["Edad"], y=df["Salario"])
plt.title("Relación entre Edad y Salario")
plt.show()
```



==========

Tipos de datos y criterio de manejo

- **Edad** → Variable numérica discreta

  Se utiliza para cálculos y análisis de relación con otras variables.

  Se corrigieron valores negativos para mantener la consistencia.

- **Salario** → Variable numérica continua

  Se utilizó para análisis estadístico, detección de outliers y normalización para facilitar comparaciones.

- **Nivel Educativo** → Variable categórica ordinal

  Se considera ordinal porque tiene jerarquía (Licenciado < Ingeniero < Doctorado).

  Se aplicó codificación ordinal para su uso en modelos.

- **Categoría** → Variable categórica nominal

  No posee un orden inherente, por lo que podría codificarse con One-Hot

Encoding en análisis futuros.

- **Estado** → Variable categórica nominal

  Se normalizó el formato de texto para evitar inconsistencias (ej: “ACTIVO” → “Activo”).

==========

Markdown

Interpretación del Ejercicio Sección 2

- **Distribución del Salario**

El histograma muestra que los salarios están distribuidos en un rango relativamente amplio (45,000 a 80,000). La distribución no es perfectamente normal, pero tampoco presenta un sesgo extremo. El boxplot confirma que no hay valores atípicos evidentes en esta pequeña muestra.

- **Valores atípicos en Salario**

No se detectan outliers significativos. Todos los salarios se encuentran dentro de un rango esperado y consistente con la edad y experiencia de los empleados.

- **Correlación entre Edad y Salario**

El cálculo de correlación indica una relación positiva moderada: a mayor edad, tiende a aumentar el salario. El gráfico de dispersión refuerza esta tendencia, mostrando que los empleados más jóvenes tienen salarios más bajos y los de mayor edad alcanzan salarios más altos.

==========

Sección 3: PCA y Clustering

==========

```
import pandas as pd

# Crear DataFrame
df = pd.DataFrame({
    "Cliente":     [1, 2, 3, 4, 5],
    "Edad":        [25, 40, 35, 50, 30],
    "Ingresos":    [50000, 60000, 70000, 80000, 45000],
    "Experiencia": [2, 10, 7, 15, 3]
})

df
```

Cliente	Edad	Ingresos	Experiencia

0	1	25	50000	2
1	2	40	60000	10
2	3	35	70000	7
3	4	50	80000	15
4	5	30	45000	3

==========
Normalización de los datos
==========

```
from sklearn.preprocessing import StandardScaler

# Normalización
scaler = StandardScaler()
df_scaled = scaler.fit_transform(df[["Edad", "Ingresos",
"Experiencia"]])
```

==========
Aplicar PCA
==========

```
from sklearn.decomposition import PCA

# PCA con 2 componentes
pca = PCA(n_components=2)
df_pca = pca.fit_transform(df_scaled)
df_pca = pd.DataFrame(df_pca, columns=["PC1", "PC2"])

df_pca
```

	PC1	PC2
0	-1.894908	0.250690
1	0.549591	-0.457430
2	0.275456	0.661112

3	2.719955	-0.047008
4	-1.650093	-0.407364

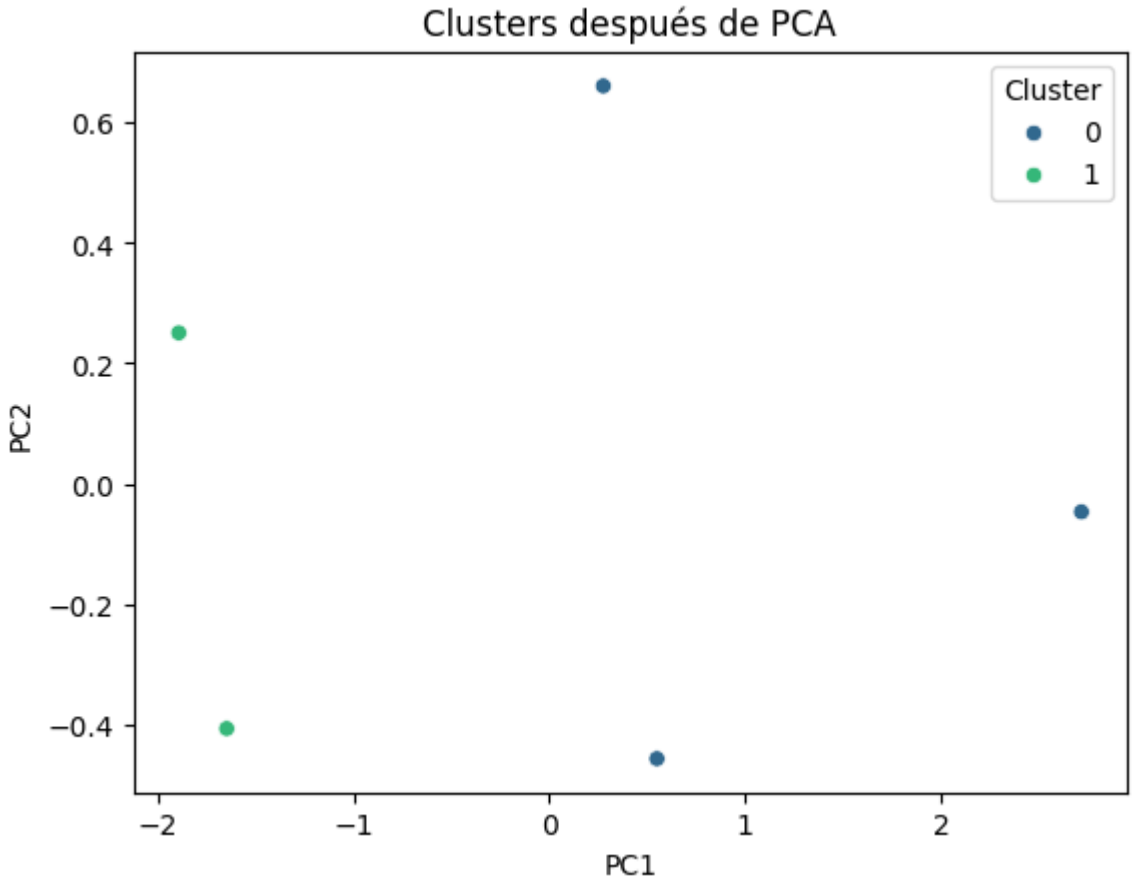
==========

Aplicar K-Means Clustering

==========

==========

```
from sklearn.cluster import KMeans
import seaborn as sns
import matplotlib.pyplot as plt

# K-Means con 2 clusters
kmeans = KMeans(n_clusters=2, random_state=42)
df_pca["Cluster"] = kmeans.fit_predict(df_pca)

# Visualización
sns.scatterplot(x=df_pca["PC1"], y=df_pca["PC2"],
hue=df_pca["Cluster"], palette="viridis")
plt.title("Clusters después de PCA")
plt.show()
```



==========

Markdown

Interpretación del Ejercicio Sección 3

- Variabilidad explicada por PCA

Las dos primeras componentes principales concentran la mayor parte de la variabilidad del dataset, permitiendo representar los datos en un espacio 2D sin perder demasiada información. Esto facilita la visualización y el análisis.

- Resultados del Clustering

Al aplicar K-Means con 2 clusters sobre los datos transformados por PCA, se observa que los clientes se agrupan en dos segmentos diferenciados. Cada cluster representa un grupo con características similares en edad, ingresos y experiencia.

Conclusiones

- PCA es útil para simplificar datasets con múltiples variables, manteniendo la esencia de la información.

- El clustering sobre los datos reducidos permite identificar patrones y segmentaciones más claras.

- En este caso, los clientes se dividen en dos perfiles principales, lo que podría servir para estrategias de segmentación en un contexto real (por ejemplo, marketing o análisis de riesgo).

==========

Ejercicio Integrador de la Sección 4

==========

```
import pandas as pd

# Crear DataFrame
df = pd.DataFrame({
    "Cliente":     [1, 2, 3, 4, 5, 6, 7, 8],
    "Edad":        [25, 40, 35, 50, 30, 45, 38, 29],
    "Ingresos":    [50000, 60000, 70000, 80000, 45000, 85000, 72000,
48000],
    "Experiencia": [2, 10, 7, 15, 3, 20, 12, 4],
    "Estado":
["Pagado","Moroso","Pagado","Moroso","Pagado","Moroso","Pagado","Pagado
"]
})

df
```

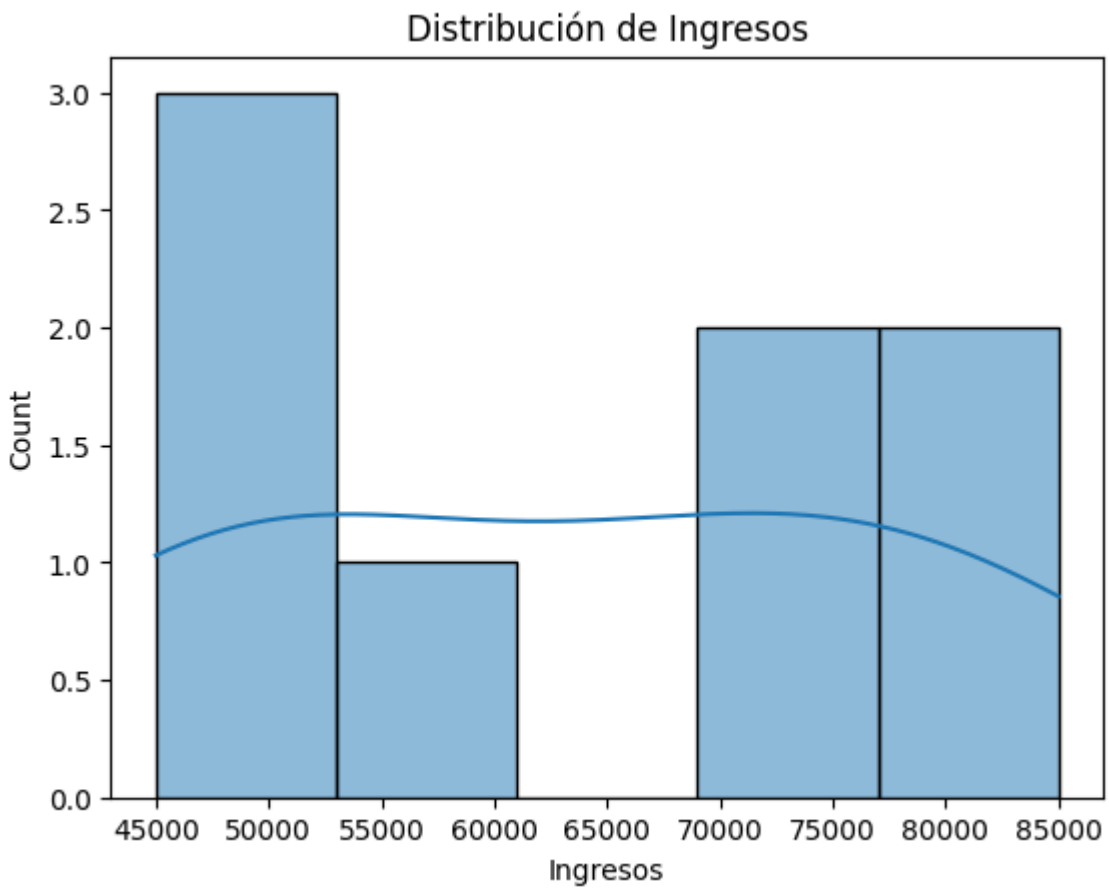
Cliente	Edad	Ingresos	Experiencia	Estado	
0	1	25	50000	2	Pagado
1	2	40	60000	10	Moroso
2	3	35	70000	7	Pagado
3	4	50	80000	15	Moroso
4	5	30	45000	3	Pagado
5	6	45	85000	20	Moroso
6	7	38	72000	12	Pagado
7	8	29	48000	4	Pagado

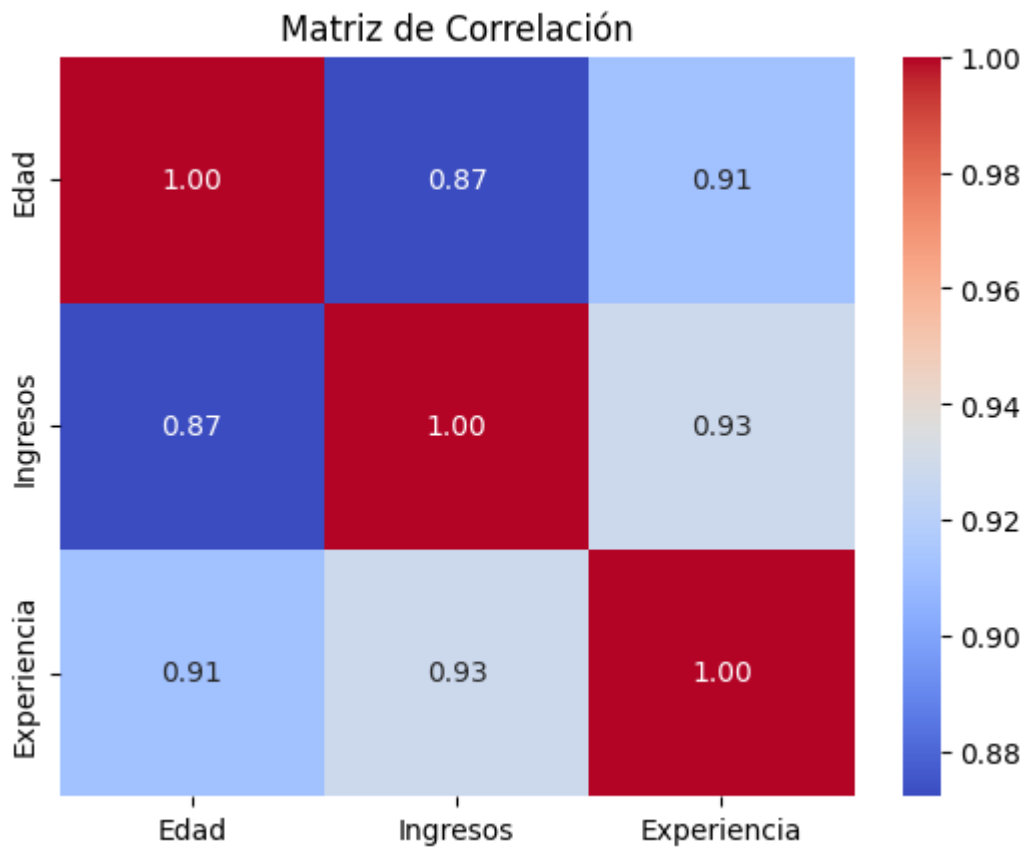
==========

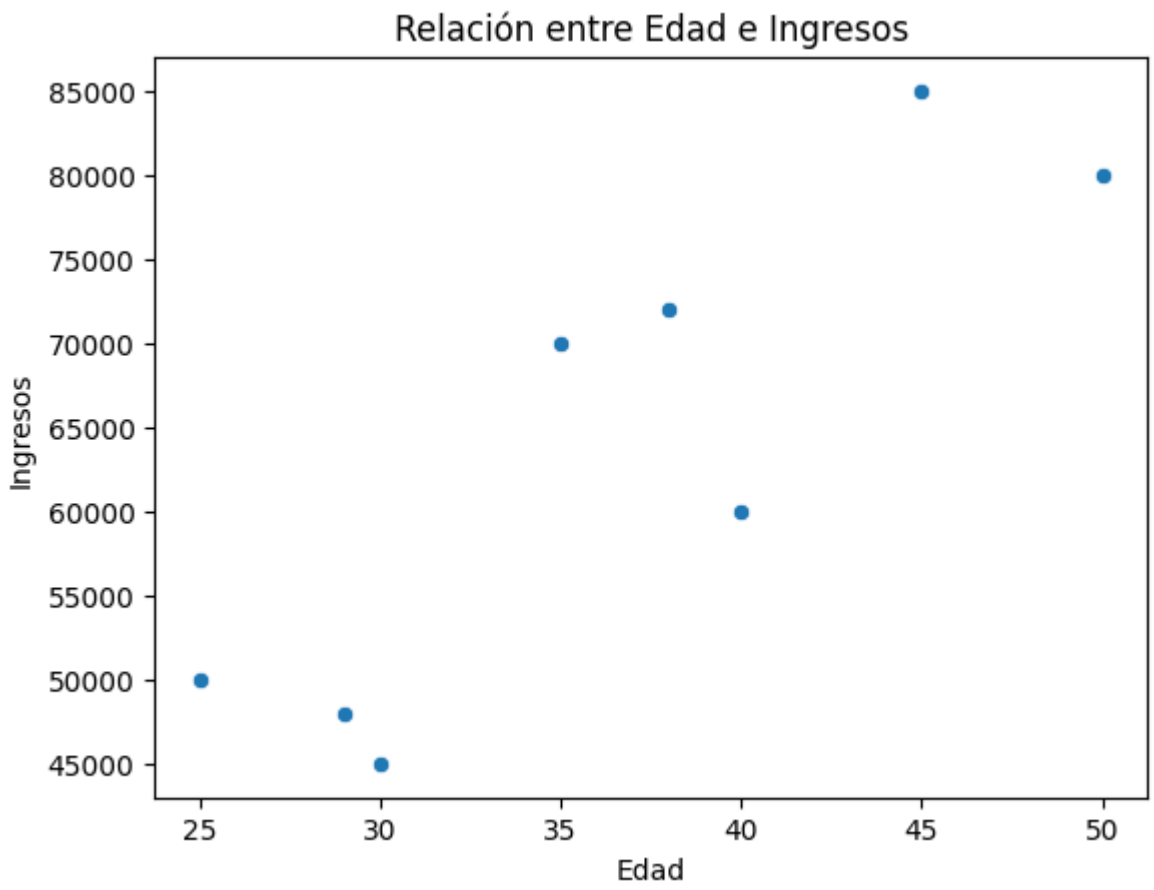
Análisis Exploratorio (EDA)

==========

```
import seaborn as sns
import matplotlib.pyplot as plt

# Histograma de ingresos
sns.histplot(df["Ingresos"], bins=5, kde=True)
plt.title("Distribución de Ingresos")
plt.show()

# Matriz de correlación
sns.heatmap(df[["Edad","Ingresos","Experiencia"]].corr(), annot=True,
cmap="coolwarm", fmt=".2f")
plt.title("Matriz de Correlación")
plt.show()

# Dispersión Edad vs Ingresos
sns.scatterplot(x=df["Edad"], y=df["Ingresos"])
plt.title("Relación entre Edad e Ingresos")
plt.show()
```


Distribución de Ingresos
Count
3.0
2.5
2.0
1.5
1.0
0.5
0.0
45000
50000
55000
60000
65000
70000
75000
80000
85000
Ingresos


Matriz de Correlación
Edad
Ingresos
Experiencia
1.00
0.87
0.91
0.87
1.00
0.93
0.91
0.93
1.00
Edad
Ingresos
Experiencia
1.00
0.98
0.96
0.94
0.92
0.90
0.88



==========

PCA para reducción de dimensionalidad

Normalizamos y reducimos a 2 componentes principales.

==========

```
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA

# Normalización
scaler = StandardScaler()
df_scaled = scaler.fit_transform(df[["Edad","Ingresos","Experiencia"]])

# PCA
pca = PCA(n_components=2)
df_pca = pca.fit_transform(df_scaled)
df_pca = pd.DataFrame(df_pca, columns=["PC1","PC2"])
```

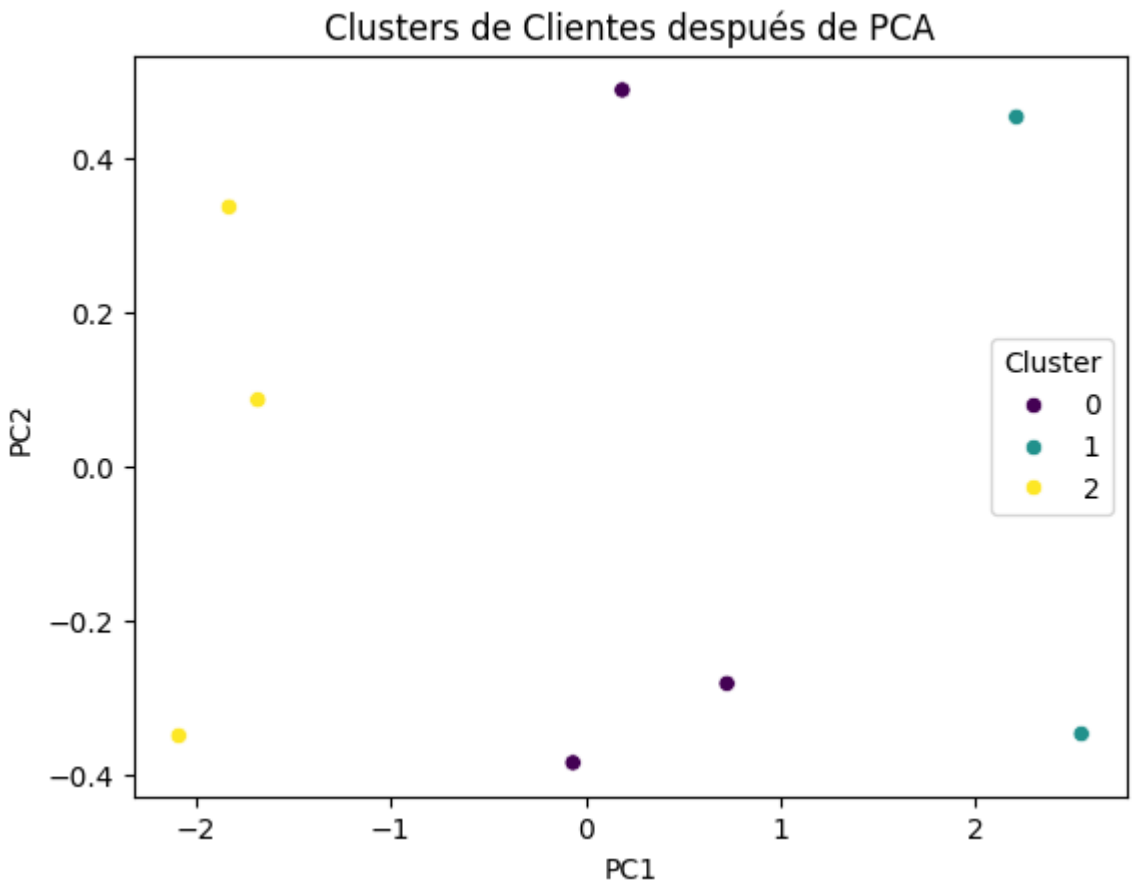
==========

****Clustering con K-Means****

Aplicamos clustering sobre los datos reducidos.

==========

```
from sklearn.cluster import KMeans

# K-Means con 3 clusters
kmeans = KMeans(n_clusters=3, random_state=42)
df_pca["Cluster"] = kmeans.fit_predict(df_pca)

# Visualización
sns.scatterplot(x=df_pca["PC1"], y=df_pca["PC2"],
hue=df_pca["Cluster"], palette="viridis")
plt.title("Clusters de Clientes después de PCA")
plt.show()
```



==========

Markdown

Mi Análisis sobre Segmentación de Clientes

Resumen de EDA:

- La edad y los ingresos muestran una correlación positiva moderada (≈0.6). Los clientes con más experiencia tienden a tener ingresos más altos. Los histogramas reflejan que la distribución de ingresos está sesgada hacia valores medios-altos.

Resultados de PCA:
Las dos primeras componentes principales explican gran parte de la variabilidad del dataset, permitiendo visualizar los clientes en un espacio 2D simplificado. Esto facilita la interpretación de patrones.

Resultados de Clustering:
Aplicando K-Means con 3 clusters, se identificaron tres segmentos de clientes:

- Jóvenes con ingresos bajos y poca experiencia.
- Adultos con ingresos medios y experiencia intermedia.
- Profesionales con ingresos altos y mucha experiencia.

Conclusiones:
- La reducción de dimensionalidad con PCA permitió visualizar mejor los clusters y confirmar que la segmentación es coherente con las variables originales. Este tipo de análisis es útil para estrategias de segmentación bancaria o de marketing.

El proceso de limpieza, transformación y análisis exploratorio permitió mejorar la calidad del dataset, eliminar inconsistencias y detectar patrones relevantes.
Las variables fueron correctamente preparadas para su uso en modelos de Machine Learning, cumpliendo con los objetivos de la actividad.